Skill Ramp-Up Plan – Part 3 (JS)

JavaScript Training Plan

Block 1 – Variables, Constants & Types

Skills

- I can declare variables with `var`, `let`, and `const`.
- I can identify and use primitive types (`string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`).
- I can check types with `typeof`.
- I can understand truthy and falsy values.

Resources

- MDN – JavaScript data types
- JavaScript.info – Variables

Definition of Done (Exercise)

- Create variables: `name` (string), `age` (number), `isStudent` (boolean).
- Assign `null` and `undefined` to two different variables and log them.
- Check the type of each variable with `typeof`.
- Test values (`0`, `""`, `NaN`, `"hello"`, `42`) inside an `if` to check if they evaluate as truthy or falsy.

Block 2 – Operators & Control Structures

Skills

- I can use arithmetic operators (+, -, *, /, %).
- I can compare values with == and ===.
- I can write conditional statements with if/else and switch.
- I can loop using for, while, and do…while.for

Resources

- MDN – Expressions and operators
- JavaScript.info – Loops

Definition of Done (Exercise)

- Write a `for` loop that prints numbers 1–20.
- Add a condition: `"even"` if divisible by 2, `"odd"` otherwise.
- Rewrite the same logic with a `while` loop.
- Create a `switch` that checks `day` and prints `"Weekend"` or `"Weekday"`.

Block 3 – Functions, Scope & Hoisting

Skills

- I can declare functions with function declarations and expressions.
- I can pass parameters and return values.
- I can explain global, function, and block scope.
- I can describe and demonstrate hoisting of variables and functions.

Resources

- MDN – Functions
- JavaScript.info – Scope

Definition of Done (Exercise)

- Write a function `sum(a, b)` that returns the sum.
- Write a function `isAdult(age)` returning `true` if `age >= 18`.
- Show scope difference with `let x = 10` inside vs outside a function.
- Use a function before declaring it (hoisting works).
- Try accessing a `var` before its declaration and compare with `let`.

Block 4 – Arrays & Objects

Skills

- I can create and manipulate arrays.
- I can use array methods like `push`, `pop`, `map`, `filter`, `reduce`.
- I can create simple objects with key/value pairs.
- I can add methods to objects and access properties with dot/bracket notation.

Resources

- MDN – Arrays
- JavaScript.info – Objects

Definition of Done (Exercise)

- Create an array `numbers = [1,2,3,4,5]`.
- Add and remove elements with `push` and `pop`.
- Use `map` to square numbers.
- Use `filter` to keep even numbers.
- Use `reduce` to sum all numbers.
- Create an object `person` with `name` and `age` and log properties.

Block 5 – References vs Copy

Skills

- I can explain copying by reference vs by value.
- I can duplicate arrays and objects safely.
- I can use spread operator and `Object.assign` for shallow copies.
- I can understand the need for deep copies.

Resources

- JavaScript.info – Object references
- MDN – Spread syntax

Definition of Done (Exercise)

- Assign `b = a` for an array and show changes in `b` affect `a`.
- Copy `a` into `c` using spread and show independence.
- Copy an object with nested properties using `Object.assign` and demonstrate shallow copy limitations.
- Create a deep copy with `structuredClone` or `JSON.parse(JSON.stringify())` and show changes don't propagate.

Block 6 – DOM Manipulation Basics

Skills

- I can select DOM elements with `querySelector`.

- I can attach event listeners.
- I can change text and styles dynamically.
- I can create and remove elements programmatically.

Resources

- MDN – DOM Introduction
- JavaScript.info – Events

Definition of Done (Exercise)

- Create a button in HTML.
- Add a click event to append `<li>` to a list.
- Change the button text to `"Clicked!"` when pressed.
- Remove the last `<li>` when another button is clicked.

Block 7 – Modern JavaScript Features

Skills

- I can write arrow functions.
- I can use default parameters, rest, and spread syntax.
- I can destructure arrays and objects.
- I can use template literals.

Resources

- MDN – ES6 Features
- JavaScript.info – Destructuring

Definition of Done (Exercise)

- Write `multiply = (a, b) => a * b`.
- Create a function `sumAll(...numbers)` that sums all inputs.
- Destructure `{ first: "Alice", last: "Smith" }`.
- Build a string with a template literal `` `Hello ${first} ${last}` ``.

Block 8 – Advanced Functions & Objects

Skills

- I can explain closures.
- I can manage `this` with `call`, `apply`, `bind`.
- I can build constructors and classes.
- I can implement inheritance and method overriding.

Resources

- JavaScript.info – Closures
- MDN – Classes

Definition of Done (Exercise)

- Implement `makeCounter()` closure with a hidden counter.
- Borrow a method using `call`.
- Create a `Car` class with `drive()`, extend into `ElectricCar` with `charge()`.
- Override `drive()` in `ElectricCar`.

Block 9 – Modules, Errors & Dynamic Imports

Skills

- I can organize code with `import/export`.
- I can use `try/catch` and throw errors.
- I can use timers with `setTimeout` and `setInterval`.
- I can dynamically import modules (lazy loading).

Resources

- MDN – Modules
- MDN – Error Handling

Definition of Done (Exercise)

- Export a function from `math.js` and import in `main.js`.
- Throw a custom error if input is not a number.
- Use `setInterval` to log time every second.
- Use `import()` to dynamically load a module only when a button is clicked.

Block 10 – Asynchronous JavaScript

Skills

- I can explain the event loop and call stack.
- I can use Promises and async/await.
- I can fetch data from APIs.
- I can handle multiple Promises with `Promise.all`.

Resources

- MDN – Promises
- JavaScript.info – Async/await

Definition of Done (Exercise)

- Write a Promise resolving after 2s.
- Use it with `await`.
- Fetch `https://jsonplaceholder.typicode.com/posts/1` and display title in DOM.
- Use `Promise.all` to fetch two posts and log both titles.

Block 11 – Advanced Data Structures & Functional Programming

Skills

- I can use `Map`, `Set`, `WeakMap`, `WeakSet`.
- I can use generators and `yield`.
- I can use Proxy and Reflect.
- I can apply functional programming (immutability, pure functions, currying, composition).
- I can use Symbols and private properties.

Resources

- MDN – Map
- JavaScript.info – Proxy

Definition of Done (Exercise)

- Create a `Set` from `[1,2,2,3]`.
- Write a generator yielding numbers 1–3.
- Use Proxy to log every property access.
- Implement `curry(sum(a,b,c))` returning a chain of calls.
- Use `compose(f,g)` to combine functions.

- Create a Symbol property on an object and show it's hidden from `for...in`.

Block 12 – Web APIs & Performance

Skills

- I can use WebSockets.
- I can register Service Workers.
- I can apply `Intl` for internationalization.
- I can optimize with debounce, throttle, lazy evaluation.

Resources

- MDN – WebSockets
- Google Developers – Performance

Definition of Done (Exercise)

- Open WebSocket to `wss://echo.websocket.org`.
- Register Service Worker caching an HTML file.
- Format today's date in French and Japanese with `Intl.DateTimeFormat`.
- Implement a search input with debounce (500ms).

Block 13 – Security & Best Practices

Skills

- I can prevent XSS with escaping.
- I can apply CSP.
- I can explain garbage collection.
- I can follow JS best practices.

Resources

- OWASP – JavaScript Security
- MDN – Memory Management

Definition of Done (Exercise)

- Fix insecure code `element.innerHTML = userInput;` with `textContent`.
- Add CSP meta tag blocking inline scripts.

- Explain with code how GC removes unreferenced objects.
- Refactor code to use `const` / `let` instead of `var`.

Block 14 – Web Components & Capstone Mini App

Skills

- I can define custom elements.
- I can use Shadow DOM for encapsulation.
- I can attach templates and styles inside components.
- I can build a Todo List app combining DOM, events, async, and localStorage.

Resources

- MDN – Web Components
- Google Developers – Web Components

Definition of Done (Exercise)

- Create a custom element `<my-card>` with Shadow DOM and styles.
- Use template literals to render its content.
- Build a Todo App where users can add/remove tasks.
- Persist tasks in `localStorage`.
- Deploy app as a single HTML+JS file with modular code.